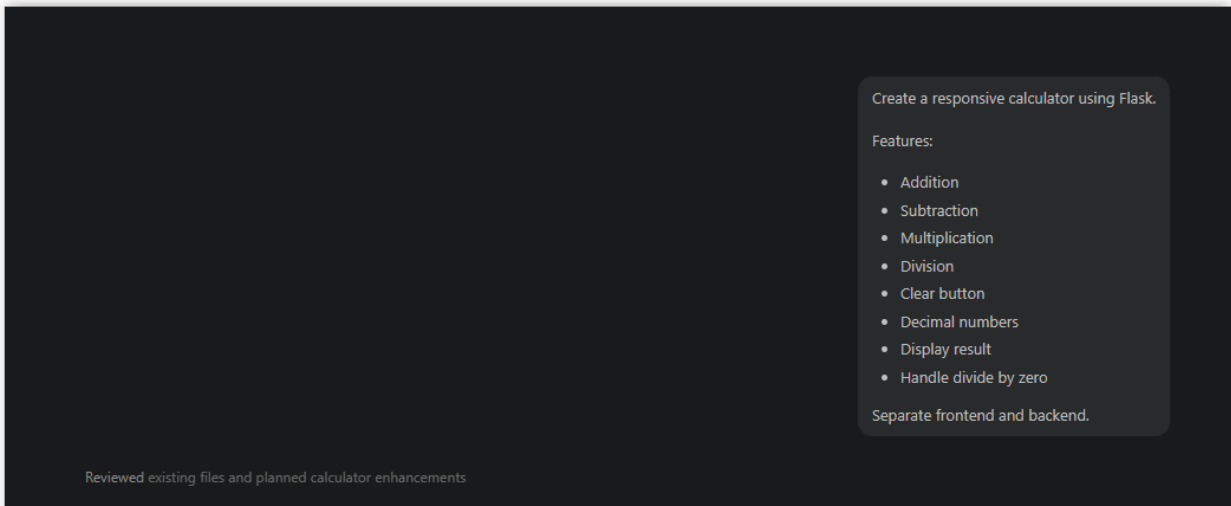
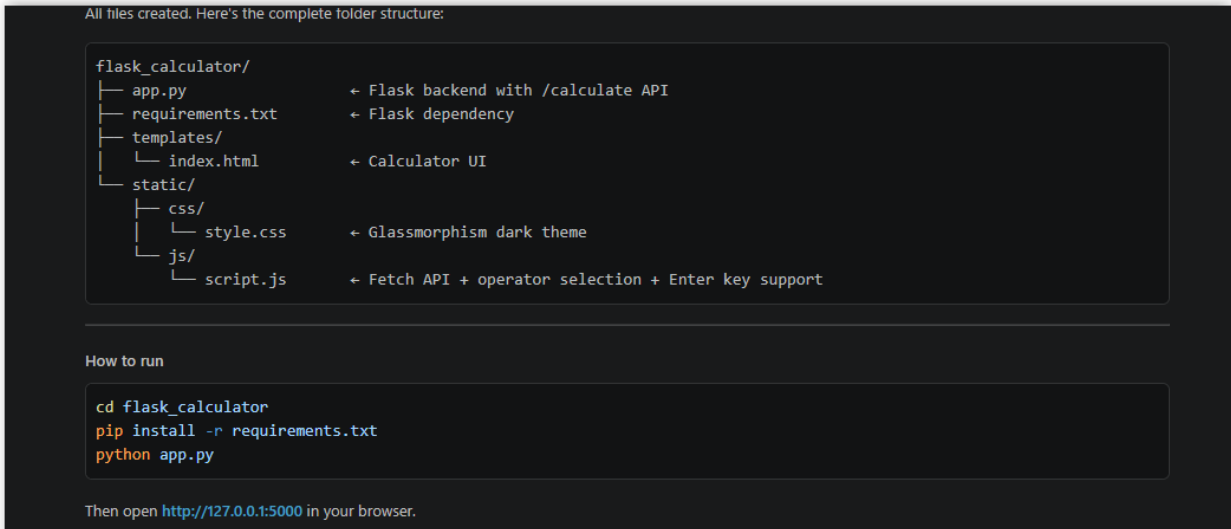
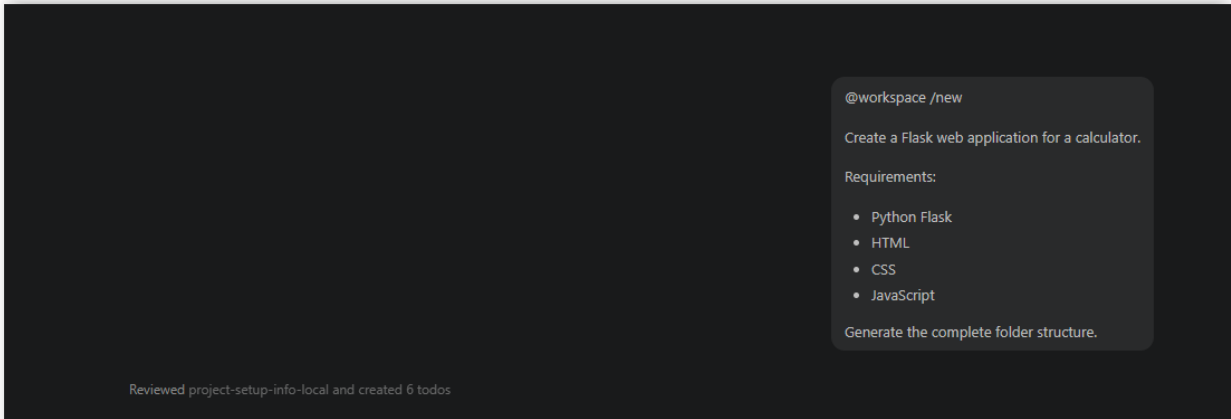




Flask backend separate.

Edited `app.py` +12 -6

Edited `index.html` +31 -12

Edited `style.css` +77 -79

Edited `script.js` +100 -44

All 4 files updated. Here's what changed:

New Calculator Layout

```
flask_calculator/
├── app.py           ← Backend: /calculate API (unchanged endpoint)
├── requirements.txt
├── templates/
│   └── index.html   ← Real button-based calculator UI
└── static/
    ├── css/style.css ← Calculator grid layout, responsive styling
    └── js/script.js  ← State machine (digit → operator → digit → =)
```

What's New

Feature	Detail
Button-based UI	Digit buttons 0-9, decimal <code>.</code> , operators <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>=</code> , <code>C</code>
Live expression display	Shows <code>12 × 3 =</code> above the result
Decimal support	<code>.</code> button, prevents double decimals
Divide by zero	Backend returns error, frontend shows message
Keyboard support	Numbers, <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>Enter</code> , <code>Escape</code>
Separated concerns	JS manages UI state; Flask backend does all math

Run

```
cd flask_calculator
python app.py
```

Open <http://127.0.0.1:5000>

Improve the CSS.

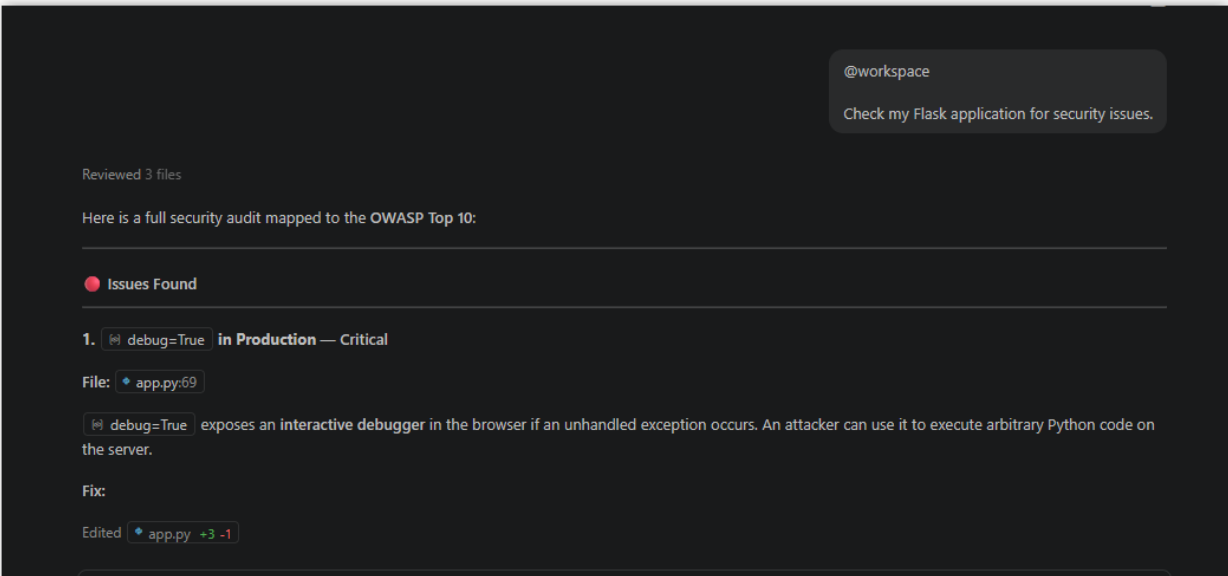
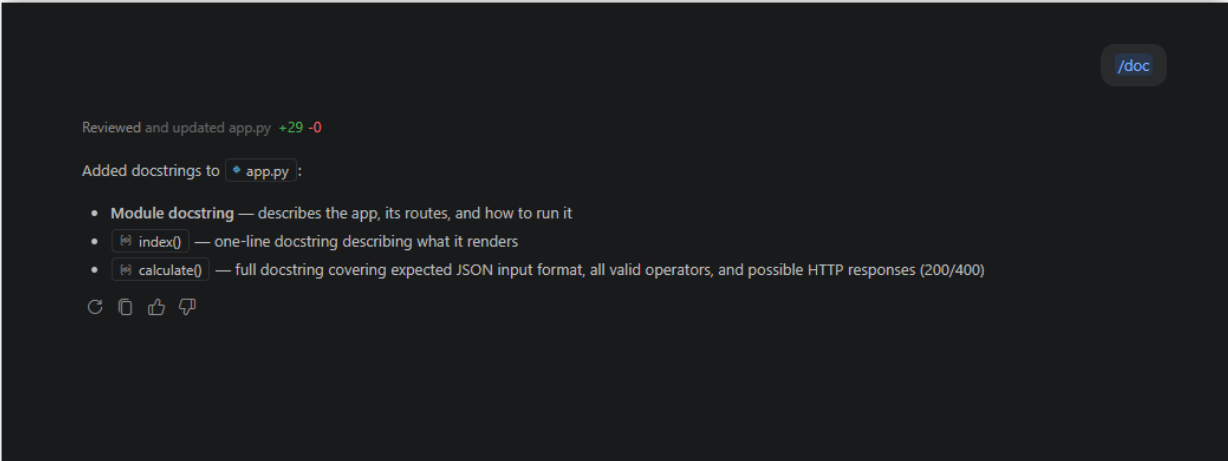
Make the calculator modern.

Use rounded buttons.

Center it on the page.

Make it responsive.

Reviewed and updated `style.css` +113 -54



File: script.js

`resultEl.textContent = ...` is used (not `innerHTML`), so user-visible output is not vulnerable to XSS. No change needed.

5. operator

not allowlisted on the backend — Medium (OWASP A03)

File: app.py

The `else` branch catches unknown operators, but only after trying all four. An explicit allowlist check up front is more secure and explicit. Already addressed via the `else → 400` path — no further change needed here, but worth noting.

Final Security Summary

#	Issue	Severity	Status
1	<code>debug=True</code> hardcoded	Critical	Fixed
2	Missing JSON body validation	High	Fixed
3	No security headers (CSP, X-Frame, etc.)	Medium	Fixed
4	XSS via <code>innerHTML</code>	N/A	Already safe (<code>textContent</code>)
5	Operator not allowlisted	Low	Already handled by <code>else → 400</code>

Issues & Suggestions

6. Missing `<meta name="description">` — SEO / Best Practice

7. No `aria-label` on buttons — Accessibility (WCAG)
Screen readers can't distinguish `+`, `-`, `×`, `÷` without labels:

`<button aria-label="Add" class="btn operator" data-op="+">+</button>`

8. `type="number"` inputs accept `e`, `+`, `-` characters — UX Bug
Users can type `1e5` or `-` into number fields. Consider `type="text"` with numeric validation in JS, or use `inputmode="decimal"`.

static/js/script.js — JavaScript

What's Good

- `textContent` used (not `innerHTML`) — XSS safe
- Enter key support
- Fetch API used correctly with `async/await`

Issues & Suggestions

#	File	Issue	Priority
1	app.py	import os inside block	Low
2	app.py	if/elif operator chain	Medium
3	app.py	Float overflow not handled	Medium
4	app.py	No rate limiting	Medium
5	app.py	Float zero comparison	Low
6	index.html	Missing meta description	Low
7	index.html	No aria-label on buttons	Medium
8	index.html	type="number" allows e, -	Medium
9	script.js	Global variable state	Low
10	script.js	Duplicate opSymbol map	Low
11	script.js	No loading state	Low
12	script.js	catch swallows error silently	Medium
13	style.css	No prefers-reduced-motion	Low
14	requirements.txt	Unpinned Flask version	Medium

What's Good

- Responsive with @media breakpoint
- min(360px, 100%) for fluid width
- Transitions and hover states present

Issues & Suggestions

13. No prefers-color-scheme or prefers-reduced-motion — Accessibility

Animations should respect user OS settings:

```
@media (prefers-reduced-motion: reduce) {  
  .btn { transition: none; }  
}
```

requirements.txt

14. No pinned version — Reproducibility

flask>=3.0.0 allows any future breaking version. Pin it:

```
flask==3.1.0
```

⚠ Issues & Suggestions

9. `selectedOperator` is a global variable — Code Quality

All logic uses loose global state. Wrapping in an IIFE or a class avoids polluting the global scope:

```
(function () {  
  let selectedOperator = null;  
  // ...all logic here  
})();
```

10. `opSymbol` `map` is defined inside the handler and duplicated — DRY

It appears in both the success handler and could be needed elsewhere. Define it once at the top:

```
const OP_SYMBOLS = { "+": "+", "-": "-", "*": "x", "/": "÷" };
```

11. No loading state on the Calculate button — UX

If the server is slow, users can spam-click. Disable the button during the fetch:

```
calculateBtn.disabled = true;  
// ... after fetch completes:  
calculateBtn.disabled = false;
```

[Review my complete project.](#)

[Suggest improvements.](#)

[Mention code quality issues](#)

Reviewed 4 files

Here is the complete project review:

Project Review — Flask Calculator

📁 `app.py` — Backend

✅ What's Good

- Security headers (`CSP`, `X-Frame-Options`, `X-Content-Type-Options`) are set
- `silent=True` on `get_json()` prevents crashes
- Division by zero is handled
- `debug=True` is no longer hardcoded

4. No rate limiting — Security

The `/calculate` endpoint accepts unlimited requests — vulnerable to abuse. Consider `flask-limiter`.

5. `num2 == 0` check is fragile — Bug Risk

`float("0.0") == 0` is `True`, but `float("-0.0") == 0` is also `True` — this is fine. However comparing floats directly is generally risky. Prefer:

```
if num2 == 0.0:
```

This is explicit and clear.

templates/index.html — Frontend

✔ What's Good

- `charset`, `viewport` meta tags present
- Static files use `url_for` (correct Flask pattern)
- Script is loaded at end of `<body>`

⚠ Issues & Suggestions

6. Missing `<meta name="description">` — SEO / Best Practice

⚠ Issues & Suggestions

1. `import os` inside `if __name__` block — Code Quality

Imports should always be at the top of the file.

2. No operator allowlist — Code Quality

The `if/elif` chain is repetitive. A dict-based dispatch is cleaner and more maintainable:

```
OPERATIONS = {
    "+": lambda a, b: a + b,
    "-": lambda a, b: a - b,
    "**": lambda a, b: a * b,
    "/": lambda a, b: a / b,
}
```

3. Float overflow not handled — Reliability

Extremely large numbers (e.g. `1e308 * 10`) produce `inf` silently. Should be caught:

```
import math
if math.isinf(result) or math.isnan(result):
    return jsonify({"error": "Result is out of range"}), 400
```

